

GROUPE

M1-LDFS

(Master 1 « Lead Dev FullStack »)

Titre Professionnel**« Expert en Informatique et Système d'Information »
(Niveau 7)**ANNEE SCOLAIRE

« 2025-2026 »

Examen partiel**- Semestre 1 -****Concevoir et développement d'une application de
« Suivi de consommation énergétique personnelle »**Contexte :

Identification & Analyse des Besoins
Cahier des charges technico-fonctionnelles
Gestion de projet et planification/suivi des tâches
Conception du système, de la BDD et de l'IHM
Développement de l'application

Proposé par

Ryadh HADJ MOKHNECHE

 ENSITECH

Attention !

- **Le travail demandé est à réaliser individuellement et non en groupe.**
- **Chaque élève devra rendre son travail individuel.**
- **La soutenance devant le jury est individuelle.**
- **L'élève devra utiliser une présentation PPT individuelle pour la soutenance devant le jury.**

Remarques !!

Prenez le temps de bien lire le sujet. Cela vous évitera d'atteindre un certain niveau de travail pour se rendre compte plus tard qu'il fallait peut-être faire les choses de manière différente.

Prenez le temps aussi de bien identifier et analyser les besoins avant de passer à la conception puis au développement.

I. Partie 1 : Description générale

I.1. Contexte du mini-projet :

Dans un contexte de transition énergétique et de hausse des coûts liés à l'énergie, les particuliers cherchent de plus en plus à **comprendre, suivre et maîtriser leur consommation énergétique** au quotidien.

Ce mini-projet consiste à concevoir et développer une **application de suivi de consommation énergétique personnelle**, permettant à un utilisateur de visualiser et analyser ses consommations (électricité, gaz, eau, ou autres sources selon les choix du groupe).

L'application devra proposer une approche claire, pédagogique et accessible, comparable à certaines solutions existantes proposées par des fournisseurs d'énergie ou des applications de suivi personnel.

I.2. Objectifs :

Ce mini-projet vise à évaluer et renforcer les compétences suivantes :

- Analyser un **besoin fonctionnel réel** et actuel ;
- Concevoir une application complète (analyse, conception, développement) ;
- Mettre en œuvre une **architecture applicative cohérente** ;
- Manipuler et visualiser des **données temporelles** ;
- Travailler en **équipe projet** sur un temps contraint ;
- Appliquer les **bonnes pratiques de développement** ;
- Intégrer les **principes de base de la sécurité applicative** ;
- Présenter et justifier des **choix techniques et fonctionnels**.

I.3. Description générale de l'application attendue

L'application développée devra permettre à un utilisateur de :

- Saisir ou importer des données de consommation énergétique ;
- Consulter ses consommations par période (jour, semaine, mois) ;
- Visualiser les données sous forme de tableaux et/ou graphiques ;
- Comparer des consommations entre différentes périodes ;
- Identifier des tendances ou évolutions de consommation.

Les données pourront être saisies manuellement, simulées, ou issues d'une source externe (API, fichiers, données mockées).

Le choix du type d'application et des technologies à utiliser sont laissés libres :

- Application **web** ou **mobile** ou **hybride**, selon les compétences et choix techniques du groupe,
- Technologies à choisir selon vos compétences.

I.4. Organisation du travail

- Le projet est à réaliser **individuellement (pas en groupe)**.
- Chaque élève est responsable de l'ensemble des phases du projet :
 - ✓ Analyse, Conception, Développement, Tests, Livraison.

I.5. Durée du projet

La durée totale du mini-projet est de **3 jours consécutifs**.

Ce délai court implique :

- une **priorisation des fonctionnalités**,
- Des choix techniques raisonnés,
- Une organisation efficace du travail en équipe.
- une capacité à livrer une application fonctionnelle dans un temps limité.

II. Partie 2 : Travail demandé

II.1. Analyse des besoins

Avant de commencer le développement, chaque groupe devra réaliser une analyse des besoins pour bien comprendre les attentes des utilisateurs et les contraintes du projet.

L'analyse doit permettre de :

- Identifier le **profil utilisateur cible** : particulier souhaitant suivre sa consommation énergétique.
- Définir les **objectifs principaux de l'application** : suivi, visualisation et analyse des consommations.
- Lister les **fonctionnalités essentielles** et les **contraintes techniques** (temps réel ou quasi temps réel, visualisation graphique, compatibilité web/mobile, sécurité des données).
- Tenir compte des **contraintes de sécurité** dès la conception : protection des données personnelles, stockage sécurisé, accès contrôlé, communications sécurisées.

Le résultat de cette analyse devra être **formalisé dans un document clair et structuré**, facilement exploitable pour la conception et le développement.

II.2. Cahier des charges fonctionnel

À partir de l'analyse des besoins, chaque groupe devra rédiger un **cahier des charges fonctionnel** décrivant précisément les fonctionnalités de l'application.

Sans être limité à cette liste, ce cahier des charges fonctionnel devra notamment inclure les fonctionnalités attendues suivantes :

- **Saisie et importation des données** : manuelle, via fichiers CSV, ou API simulée.
- **Consultation des consommations** par période (jour, semaine, mois).
- **Visualisation des données** : tableaux et graphiques interactifs.
- **Comparaison de périodes** pour identifier les tendances et économies possibles.
- **Alertes ou notifications** si consommation inhabituelle (optionnel mais encouragé).
- **Navigation claire** entre les différentes sections et fonctionnalités.

Chaque fonctionnalité devra être décrite avec précision : objectif, comportement attendu, interactions avec l'utilisateur et contraintes éventuelles.

II.3. Cahier des charges technique

Chaque groupe devra également produire un **cahier des charges technique** décrivant les choix techniques réalisés pour le projet.

Ce document devra préciser notamment :

II.3.1. Choix technologiques

- Type d'application : web, mobile ou hybride ;
- Technologies utilisées :
 - ✓ Frontend : framework ou bibliothèque choisi.
 - ✓ Backend : serveur, API, logique applicative.
 - ✓ Base de données : type et structure.
- Mode de récupération des données :
 - ✓ Saisie manuelle,
 - ✓ Import de fichiers,
 - ✓ API simulée ou réelle.

Les choix devront être **justifiés** en tenant compte du temps imparti et du niveau de complexité acceptable.

II.3.2. Architecture générale

- Organisation front-end / back-end.
- Gestion des échanges de données.
- Choix pour la mise à jour des données en quasi temps réel (polling, WebSocket, autre).

II.3.3. Prise en compte de la sécurité

Le cahier des charges devra inclure les principes de **sécurité applicative** adaptés au mini-projet :

- **Protection des données utilisateur** : chiffrement des informations sensibles, accès restreint si nécessaire.
- **Communications sécurisées** : HTTPS pour les échanges client/serveur.
- **Validation et contrôle des données** : filtrage des saisies utilisateur, prévention d'injections ou d'erreurs critiques.
- **Gestion des fichiers importés** : vérification du format et des contenus.

L'objectif est de démontrer une **conscience et une mise en œuvre des bonnes pratiques de sécurité**, même sur un projet de courte durée.

II.4. Gestion de projet et organisation du travail

Pour réussir ce mini-projet dans un délai court, chaque élève devra adopter une **organisation de travail structurée**, inspirée des bonnes pratiques de gestion de projet informatique.

L'objectif est de **planifier, suivre et organiser les tâches** afin de garantir l'avancement du projet et la qualité des livrables.

II.4.1. Méthodologie de gestion de projet (Agile / Scrum)

Le mini-projet devra être conduit **individuellement**, en s'appuyant sur une **méthodologie de gestion de projet adaptée au travail en autonomie**.

Une approche **Agile / Scrum simplifiée** est recommandée.

L'étudiant devra notamment :

- Définir ses **objectifs quotidiens** (mini-sprints journaliers) ;
- Planifier les fonctionnalités à réaliser sur les **3 jours du projet** ;
- Organiser son travail de manière itérative (analyse → conception → développement → tests) ;
- Être capable d'**ajuster ses priorités** en fonction de l'avancement et des difficultés rencontrées.

Même en travail individuel, l'étudiant devra démontrer une **démarche structurée**, proche d'un contexte professionnel.

II.4.2. Planification et suivi des tâches (Kanban)

Chaque étudiant devra mettre en place un **outil de planification et de suivi des tâches** basé sur un tableau **Kanban**, à l'aide d'un outil tel que **Trello**, **Jira** ou équivalent.

Le tableau Kanban devra comporter au minimum :

- **À faire** ;
- **En cours** ;
- **Terminé**.

L'étudiant devra :

- Découper le projet en **tâches claires et concrètes** (analyse, conception, développement front-end, back-end, tests, documentation) ;
- Estimer la charge de chaque tâche (même de manière approximative) ;
- Mettre à jour régulièrement le tableau pour refléter l'état réel du projet ;
- Utiliser le Kanban comme **outil d'aide à la priorisation et à l'auto-organisation**.

Le tableau devra montrer une **progression logique et réaliste** sur les 3 jours.

II.4.3. Gestion du code source (GitHub / GitLab)

Le projet devra obligatoirement être versionné à l'aide d'un **outil de gestion de code source** tel que **GitHub** ou **GitLab**, même dans un contexte de travail individuel.

Les bonnes pratiques suivantes devront être respectées :

- Création d'un **dépôt de projet** dès le début ;
- Réalisation de **commits réguliers**, correspondant à des avancées logiques ;
- Rédaction de **messages de commit clairs et explicites** ;
- Organisation cohérente du dépôt (arborescence, séparation front-end / back-end si applicable) ;
- Rédaction d'un **README** présentant :
 - L'objectif du projet,
 - Les technologies utilisées,
 - Les instructions de lancement de l'application.

L'utilisation de branches est encouragée, mais non obligatoire dans le cadre d'un travail individuel.

II.4.4. Traçabilité et justification des choix

L'étudiant devra être en mesure de démontrer :

- La **traçabilité de son travail** sur l'ensemble du projet ;
- La cohérence entre la planification initiale et le travail réellement effectué.

Il devra notamment être capable de :

- Expliquer l'évolution de son tableau Kanban ;
- Commenter l'historique des commits Git ;
- Justifier ses choix :
 - Fonctionnels,
 - Techniques,
 - Organisationnels.

Cette partie vise à évaluer la **capacité de l'étudiant à adopter une posture professionnelle en autonomie**, comparable à celle attendue dans un contexte de développement individuel ou de mission en entreprise.

II.5. Conception de l'application

Avant la phase de développement, chaque étudiant devra réaliser une conception complète et structurée de l'application.

Cette conception servira de référence tout au long du développement et devra être cohérente avec les fonctionnalités effectivement implémentées.

La conception est organisée en trois volets :

1. Conception système (UML),
2. Conception de la base de données relationnelle (MERISE),
3. Conception du front-end (UX / UI).

II.5.1. Conception du système (UML)

Chaque étudiant devra produire une **conception système** à l'aide de diagrammes UML, permettant de comprendre le fonctionnement global de l'application.

a) *Diagramme de cas d'utilisation (Use Case)*

Le diagramme devra :

- Identifier les acteurs (utilisateur non authentifié, utilisateur authentifié) ;
- Décrire les cas d'utilisation principaux, notamment :
 - ✓ S'inscrire ;
 - ✓ S'authentifier ;
 - ✓ Consulter ses consommations ;
 - ✓ Ajouter une consommation ;
 - ✓ Visualiser l'évolution de la consommation.

b) Diagrammes de séquence

Des **diagrammes de séquence détaillés** devront être réalisés pour :

- Inscription
- Authentification
- Fonctionnalités métier (**au minimum 2**) : les fonctionnalités métier suivantes sont imposées :
 - **Ajout d'une consommation énergétique**
 - **Consultation et visualisation des consommations**

Les diagrammes devront mettre en évidence les interactions entre :

- L'interface utilisateur,
- La logique applicative,
- La base de données ou l'API.

c) Diagrammes d'activité

Des diagrammes d'activité devront illustrer :

- Le parcours utilisateur pour l'inscription et l'authentification ;
- Le flux principal pour au moins une fonctionnalité métier.

d) Diagramme de classes

Le diagramme de classes devra :

- Identifier les classes principales (Utilisateur, Consommation, Énergie, etc.) ;
- Définir les attributs essentiels ;
- Représenter les relations entre classes ;
- Être cohérent avec la logique métier et la base de données.

e) Diagramme des entités

Un diagramme des entités devra présenter :

- Les entités métier ;
- Leurs relations ;
- Les cardinalités principales.

II.5.2. Conception de la base de données relationnelle (Méthode MERISE)

Chaque élève devra réaliser une **conception de la base de données** en s'appuyant sur la méthode **MERISE**.

a) Modèle Conceptuel de Données (MCD)

- Identification des entités (*Utilisateur, Consommation, TypeEnergie*, etc.) ;
- Associations et cardinalités ;
- Attributs principaux.

b) Modèle Logique de Données (MLD)

- Transformation du MCD en tables relationnelles ;
- Définition des clés primaires et étrangères ;
- Respect des règles de normalisation.

c) Modèle Physique de Données (MPD)

- Adaptation au SGBD choisi ;
- Types de données ;
- Contraintes d'intégrité.

d) Dictionnaire de données

Pour chaque table du MPD réaliser un dictionnaire de données

- Description de chaque donnée :
 - ✓ Nom, type, taille, min, max, description, contraintes éventuelles, format.
- Cohérence avec les modèles précédents.

L'ensemble devra être clair, structuré et directement exploitable pour l'implémentation.

II.5.3. Conception de l'interface utilisateur (Front-end)

Chaque élève devra réaliser une **conception du front-end** afin de définir l'expérience utilisateur et l'aspect visuel de l'application.

La conception devra obligatoirement couvrir :

- Les pages d'inscription et d'authentification ;
- Au minimum une page liée à une fonctionnalité métier, par exemple :
 - Ajout d'une consommation,
 - Consultation des consommations avec graphique.

a) Zonings

- Découpage des écrans principaux en zones fonctionnelles ;
- Identification des éléments clés (navigation, contenu, informations de match, etc.).

b) Wireframes

- Représentation simplifiée des écrans ;
- Organisation des informations ;
- Parcours utilisateur.

c) Charte graphique

- Choix des couleurs ;
- Typographies ;
- Icônes ;
- Cohérence visuelle globale.

d) Maquettes graphiques

- Maquettes plus détaillées des écrans principaux ;
- Respect de la charte graphique ;
- Lisibilité et ergonomie.

Les outils de conception sont laissés au choix des groupes (ex. : Figma, Adobe XD, papier/crayon numérisé, etc.).

II.5.4. Cohérence globale de la conception

L'ensemble des conceptions devra être :

- **Cohérent entre les différents volets** (UML ↔ BDD ↔ Front-end) ;
- Adapté aux fonctionnalités réellement développées ;
- Compréhensible par un tiers.

La qualité de la conception sera évaluée autant que le résultat fonctionnel final.

II.6. Développement de l'application

Chaque élève devra réaliser le **développement complet de l'application**, en s'appuyant sur les phases de conception précédemment définies.

Le développement devra couvrir :

- Le **front-end** (interface utilisateur),
- Le **back-end** (logique applicative, accès aux données),
- La **base de données** (si utilisée).

L'application développée devra être fonctionnelle, stable et cohérente avec les documents de conception.

II.6.1. Développement Front-end

Le développement front-end devra permettre à l'utilisateur de :

- S'inscrire et s'authentifier ;
- D'accéder au moins à une fonctionnalité métier après authentification. Exemple :
 - Consulter et manipuler ses données de consommation énergétique ;
 - Visualiser les données sous forme lisible (tableaux, graphiques).

a) Exigences techniques

Les étudiants devront :

- Respecter la structure définie lors de la conception ;
- Organiser le code de manière claire et maintenable ;
- Séparer la logique métier de l'affichage ;

b) Sécurité côté Front-end

Le front-end devra intégrer les **bonnes pratiques de sécurité** suivantes :

- Validation des données affichées et reçues ;
- Prévention des failles courantes côté client (ex. : XSS basiques) ;
- Gestion propre des erreurs sans divulgation d'informations sensibles ;
- Absence de clés d'API ou de secrets directement exposés dans le code client.

II.6.2. Développement Back-end

Le développement back-end devra assurer :

- La gestion des utilisateurs (inscription, authentification) ;
- Et au moins une fonctionnalité métier. Exemple :
 - La gestion des données de consommation énergétique ;
 - La communication sécurisée avec le front-end via une API.

a) API et logique applicative

Le back-end devra :

- Exposer des *endpoints* clairs et cohérents ;
- Respecter une structure logique (routes, services, contrôleurs, etc.) ;
- Gérer les erreurs et les cas particuliers ;
- Contrôler les données échangées avec le front-end.

c) Sécurité côté Back-end

Une attention particulière devra être portée à :

- La validation et le filtrage des données entrantes ;
- Le hachage des mots de passe avant stockage en base de données ;
- La gestion sécurisée de l'authentification (sessions, tokens, ou mécanisme équivalent) ;
- La protection des endpoints contre les accès non autorisés ;
- L'utilisation de communications sécurisées (HTTPS).

II.6.3. Gestion et sécurisation de la base de données

Si une base de données est utilisée, celle-ci devra être :

- Conforme au modèle de données MERISE conçu ;
- Correctement structurée et sécurisée.

Les bonnes pratiques suivantes sont attendues :

- Utilisation de requêtes sécurisées (prévention des injections SQL) ;
- Définition de contraintes d'intégrité (clés, relations) ;
- Limitation des droits d'accès à la base de données ;
- Protection des données stockées (pas de données inutiles ou sensibles).

II.6.4. Sécurité globale de l'application

La sécurité devra être prise en compte de manière **transversale** tout au long du développement.

Chaque groupe devra démontrer :

- Une compréhension des **risques de sécurité basiques** ;
- Une cohérence entre les choix techniques et les mesures de protection mises en place ;
- Une application fonctionnelle sans failles évidentes.

L'objectif n'est pas de mettre en œuvre une sécurité avancée, mais d'appliquer des **principes fondamentaux de sécurité applicative** adaptés à un mini-projet.

II.6.5. Tests et vérifications

Avant la livraison finale, l'élève devra :

- Tester les fonctionnalités principales ;
 - Inscription,
 - Authentification,
 - Gestion des consommations ;
- Vérifier la cohérence entre front-end, back-end et base de données ;
- S'assurer que les données sont correctement protégées et accessibles uniquement par l'utilisateur concerné.

Les tests pourront être manuels et ciblés, en lien avec les fonctionnalités développées, mais devront être **réfléchis et justifiés..**

Livrables attendus :

Chaque élève devra fournir les **livrables suivants**, organisés de manière claire et structurée.

➤ Livrables de conception :

- Analyse des besoins
- Cahier des charges fonctionnel
- Cahier des charges technique
- Documents de conception :
 - UML : diagrammes de cas d'utilisation, de séquence, d'activité, de classes, des entités ;
 - MERISE : MCD, MLD, MPD, dictionnaire de données ;
 - Conception front-end : zonings, wireframes, charte graphique, maquettes.

➤ Livrables de gestion de projet

- Description de la méthodologie de gestion de projet utilisée (ex. Agile / Scrum)
 - Tableau Kanban (Trello, Jira ou équivalent) montrant : les tâches, leur état d'avancement, et la répartition du travail
 - Dépôt GitHub / GitLab avec : historique des commits et organisation du projet

➤ Livrables de développement

- Code source complet de l'application (front-end et back-end)
- Application fonctionnelle (locale ou déployée si possible)
- Respect des principes de sécurité abordés dans le sujet

Modalités de rendu :**➤ Rendu des livrables**

Les livrables devront être remis selon les modalités précisées par le formateur :

- lien vers le dépôt GitHub / GitLab,
- documents de conception (PDF ou équivalent),
- lien vers l'application ou démonstration locale.

➤ Présentation orale individuelle

Chaque élève devra réaliser une **présentation orale individuelle** d'une durée de **30 minutes**.

Ensuite, le jury effectuera un **entretien technique** (sous forme d'échanges et questions) d'une durée de **30 minutes**.

- Chaque élève **devra utiliser son propre support PowerPoint**, réalisé individuellement.
- Chaque élève devra être capable d'expliquer les choix fonctionnels, les choix techniques, la conception, la gestion de projet, la sécurité, sa contribution personnelle au projet.

La présentation pourra s'appuyer sur une démonstration de l'application, des diagrammes de conception et de l'architecture, des captures du Kanban, des extraits du dépôt Git.

Critères d'évaluation :

L'évaluation portera à la fois sur :

- La **qualité du travail individuel**,
- La **compréhension individuelle** du projet,
- La **posture professionnelle** adoptée.

Les critères principaux sont :

- Compréhension du besoin et pertinence de l'analyse ;
- Qualité et cohérence des conceptions (UML, MERISE, front-end) ;
- Qualité du développement (fonctionnalités, structure, lisibilité) ;
- Prise en compte de la sécurité ;
- Organisation et gestion de projet ;
- Qualité de la présentation orale individuelle.

